

POO COM JAVASCRIPT

Pedro Henrique Ribeiro Dias

Inatel

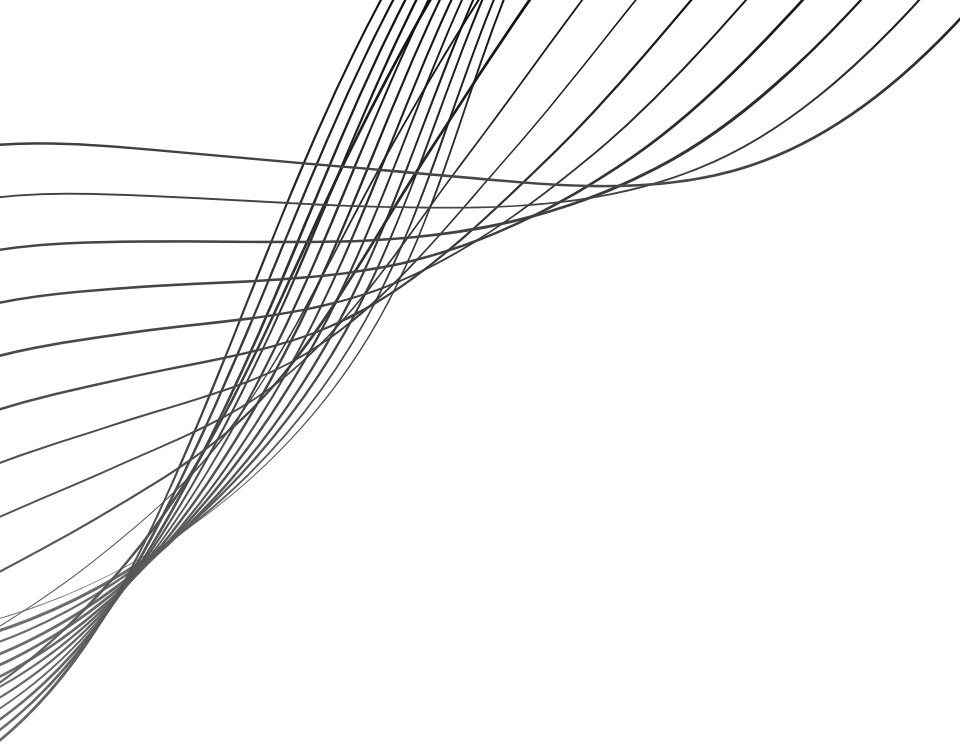
INTRODUÇÃO

JavaScript: A Linguagem da Web Viva

Diferente de outras linguagens onde o ambiente de execução é previsível e controlado, o JavaScript nasceu no caos – criado em 10 dias para rodar dentro de um navegador e reagir ao que o usuário faz em tempo real. Hoje ele é a única linguagem que roda nativamente no navegador, domina o backend com Node.js e ainda sustenta aplicações mobile e desktop.

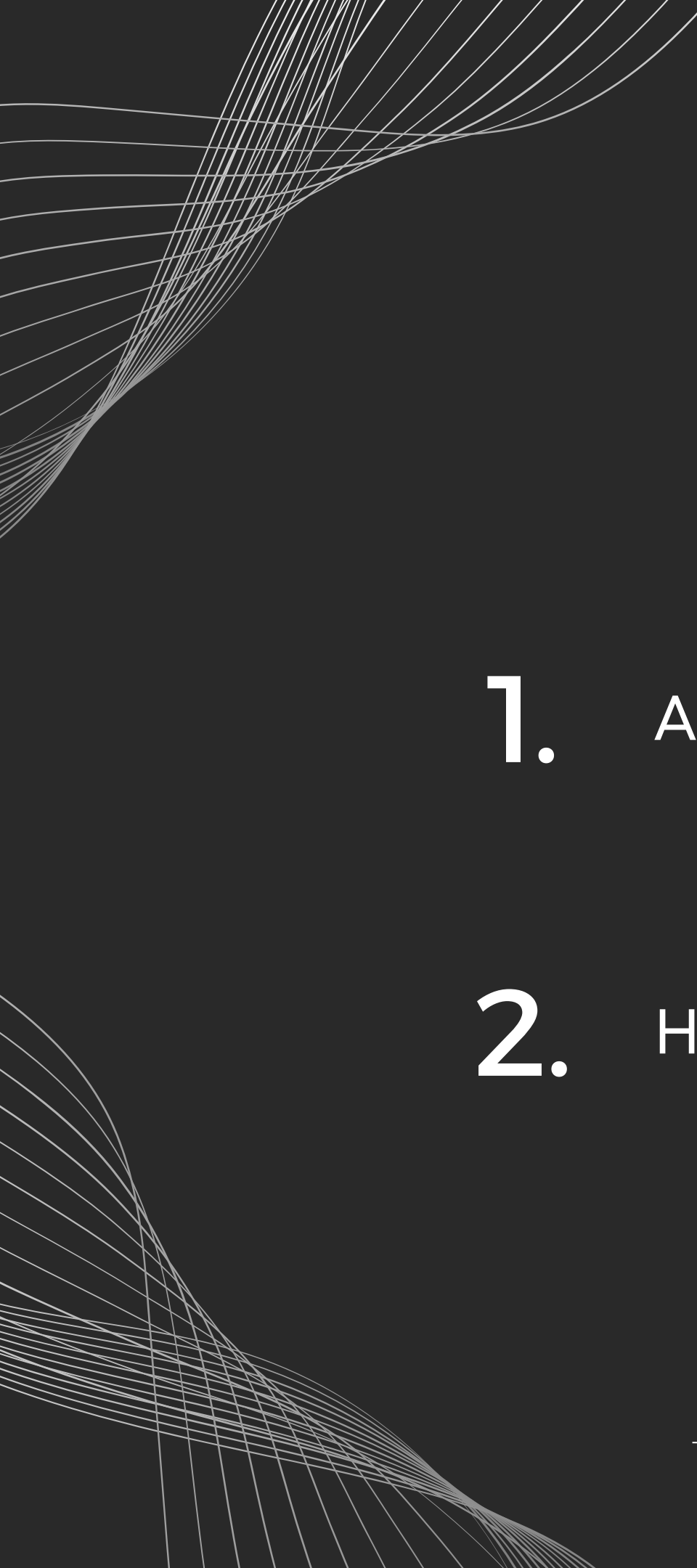
O que torna o JavaScript único?

- Onipresença: É a única linguagem que vive ao mesmo tempo no frontend e no backend, permitindo que um único desenvolvedor construa sistemas completos do zero.
- Dinamismo: JavaScript é de tipagem dinâmica e altamente flexível – objetos podem ser criados e modificados em tempo de execução, oferece uma liberdade que poucas linguagens permitem.
- Assincronismo: O JavaScript foi construído para não travar. Através de callbacks, Promises e async/await, ele lida com operações demoradas sem parar a execução –essencial para aplicações web modernas



INTRODUÇÃO

"Entender JavaScript é aceitar que a linguagem foi feita para o imprevisível. Aqui, mais do que seguir regras, você aprende a domar o comportamento de sistemas vivos."



PILARES DA POO

1. Abstração

2. Herança

3. Encapsulamento

4. Polimorfismo

RELACIONAMENTOS

- Composição: A classe cria e controla o objeto interno.
- Agregação: A classe recebe o objeto pronto.

Ou seja:

- ◆ Composição → dependência total
- ◆ Agregação → ligação fraca (referência)

CONSTRUTORES E COLLECTIONS

1. JavaScript possui diversas estruturas para armazenar objetos:
 - ◆ Array, Map, SetE métodos para manipular coleções:
 - ◆ .forEach(), .map(), .filter(), .find()

INTERFACES

Interfaces em JavaScript são simuladas via classes abstratas

- JavaScript não tem interfaces formais nem classes abstratas nativas.
- O contrato é criado lançando um erro no método base com `throw new Error()`.
- Classes filhas herdam e sobrescrevem o método – se não sobrescreverem, o erro aparece em tempo de execução.

EXERCÍCIO 1

Cenário: Em uma batalha Pokémon, cada criatura tem seus próprios atributos e um estilo único de atacar. A vida de um Pokémon não pode ser alterada diretamente de fora só o próprio combate pode mudá-la.

Tarefa:

- Classe base: Crie a classe Pokemon com os atributos nome e tipo, e um atributo interno `_vida` que só pode ser lido de fora via getter. Implemente o método atacar com um comportamento genérico.
- Herança e Sobrescrita: Crie as classes Pikachu e Charizard herdando de Pokemon. Cada uma deve sobrescrever o método atacar com seu próprio comportamento.
- Encapsulamento: Implemente o método `receber_dano` na classe base para reduzir a vida de forma controlada. A vida não pode ficar negativa.
- Main: Instancie um Pikachu e um Charizard, faça cada um atacar e em seguida aplique dano a um deles via `receber_dano`. Exiba a vida restante usando o getter.

○

EXERCÍCIO 2

Cenário: Um time de Valorant é formado por Agentes, cada um com uma função tática e uma habilidade característica do seu papel. Todo Agente precisa cumprir o contrato de combate para entrar na partida.

Tarefa:

- **Interface** (simulação de interface): Crie a classe base Habilidade com o método usar que lança um erro caso não seja sobrescrito. Crie as classes de habilidades dos agentes: Smoke, Flash, Dash e Armadilha herdando de Habilidade, cada uma sobrescrevendo usar com seu próprio comportamento.
- **Composição:** Crie a classe Agente com os atributos nome e funcao (ex: "Controlador", "Iniciador", "Duelista", "Sentinela"). No constructor, receba um objeto do tipo Habilidade e armazene como atributo interno do Agente. Implemente o método entrarEmCombate que exibe o nome, a função e chama usar da sua habilidade.

EXERCÍCIO 2

- **Agregação:** Crie a classe Time que recebe um array de Agentes já criados. Implemente o método iniciarPartida que percorre todos os agentes com forEach e chama entrarEmCombate de cada um.
- Implemente o método listarControladores no Time que usa filter e instanceof para retornar apenas os Agentes cuja habilidade é do tipo Smoke.
- **Main:** Instancie pelo menos um Agente de cada função – sugestões: Omen com Smoke, Skye com Flash, Jett com Dash e Cypher com Armadilha. Agrupe-os em um Time, chame iniciarPartida e em seguida exiba apenas os controladores encontrados pelo listarControladores.

EXERCÍCIO 3

Cenário: Em Gravity Falls, Dipper carrega o misterioso Diário 3, que guarda registros sobre as criaturas sobrenaturais da cidade. A Cabana do Mistério, por sua vez, tem seus próprios personagens que trabalham e vivem por lá.

Tarefa:

- **Composição:** Crie a classe **Criatura** com os atributos **nome** e **perigo**. Crie a classe **Diario** que no constructor instancia internamente um array de criaturas. Implemente o método **registrarCriatura** que recebe uma **Criatura** e a adiciona ao array, e o método **listarCriaturas** que exibe todas as criaturas registradas.
- **Agregação:** Crie a classe **Personagem** com o atributo **nome**. Crie a classe **CabanaMisterio** que recebe um array de **Personagens** já criados no constructor. Implemente o método **listarFuncionarios** que percorre o array com **forEach** e exibe cada personagem.
- **Main:** Instancie um **Diario**, crie duas ou três **Criaturas** e registre-as nele. Em seguida instancie alguns **Personagens** como **Grunkle Stan**, **Mabel** e **Soos**, agrupe-os em uma **CabanaMisterio** e chame **listarFuncionarios**.

EXERCÍCIO 4

Cenário: A Associação Hunter recebeu um alerta: a Trupe Fantasma foi avistada em Yorknew City. Gon, Killua e Kurapika precisam se organizar para enfrentar as Aranhas. Cada Hunter tem seu próprio estilo de luta e cabe a você modelar esse confronto.

Tarefa:

- Classe base: Crie a classe Hunter com o atributo nome e um método lutar com comportamento genérico.
- Herança e Sobrescrita: Crie as classes Gon, Killua e Kurapika herdando de Hunter. Cada uma deve sobrescrever o método lutar com seu próprio estilo de combate – sugestões: Gon com Jajanken, Killua com Godspeed e Kurapika com suas correntes.
- Agregação: Crie a classe TrupeFantasma com o atributo nome e um array de membros passado no constructor. Implemente o método revelarMembros que percorre o array com forEach e exibe cada membro.
- Main: Instancie os três Hunters e faça cada um chamar lutar. Monte uma TrupeFantasma com dois ou três membros como Chrollo, Hisoka e Feitan, e chame revelarMembros.

FIM DA AULA!



zSh3rl0cK/S01-monitoria

Contribute to zSh3rl0cK/S01-monitoria development by creating an account on GitHub.



GitHub